ITM SKILLS UNIVERSITY

FINAL PROJECT REPORT

Bachelor of Technology in Computer Science and Engineering

Class BTech CSE 25–29

PROJECT TITLE

Travel Agency Customer and Booking Database Using SQL

Case Study 155: Travel Agency Booking Management

Submitted By	Rashmeet Kaur Nanade
Roll No:	150096725165
Cohort:	Demis Hassabis
Subject	DBMS-SQL & NOSQL

1. Project Overview

The Travel Agency Customer and Booking Database is a relational database designed and implemented in MySQL to manage the domestic and international bookings of a travel agency (Case Study 155: Travel Agency Booking Management). The primary syllabus focus of the case study is UNION, UNION ALL and set operations.

The agency keeps domestic bookings and international bookings in two separate tables for operational reasons, while management needs combined reports across both datasets. The main purpose of this project is to show how SQL set operators and related queries can combine compatible query results from the two tables without merging the tables themselves.

The system stores each booking with the customer name, destination, booking date and amount. Primary keys, AUTO_INCREMENT and NOT NULL constraints are used to maintain data integrity. The project demonstrates the following DBMS concepts:

- Relational database design, entities and attributes
- Primary keys, AUTO_INCREMENT and NOT NULL constraints
- SQL data types: INT, VARCHAR(100), DATE and DECIMAL(10,2)
- SQL DDL, DML and SELECT queries
- Set operations: UNION and UNION ALL
- INNER JOIN with DISTINCT to find common customers
- Derived-table subquery with an alias
- GROUP BY, COUNT(*) and aggregate functions
- Result verification using SHOW TABLES, SELECT DATABASE() and COUNT(*)

The project provides a structured way to produce combined booking reports and shows how SQL can solve a practical reporting problem.

2. Problem Statement

A travel agency maintains domestic and international customer bookings separately for operational reasons. Management wants combined reports across both datasets. The system must:

- Store domestic bookings and international bookings in separate tables with customer, destination, booking date and amount.
- Combine the customers of both datasets into a single report.
- Preserve duplicate customer entries when every booking has to be counted.
- Identify customers who appear in both the domestic and international datasets.
- Display all destinations in a single report.
- Compare the number of bookings made by each customer.
- Verify that the database, tables and record counts are correct.

3. Project Objectives

- To create the domestic_bookings and international_bookings tables containing customer, destination, booking date and amount.
- To insert sample data: domestic destinations Goa, Jaipur and Kerala; international destinations Dubai, Singapore and Bangkok; customers Asha, Vikram, Rohan and Meera.
- To combine domestic and international customers using UNION.
- To use UNION ALL to preserve duplicates.
- To identify customers appearing in both datasets.
- To display all destinations in a single report.
- To compare customer booking counts using UNION ALL, COUNT(*) and GROUP BY.
- To verify the final results using the terminal outputs.

4. Technologies and Tools Used

Component	Technology
Database	MySQL (installed using Homebrew)
Query Language	SQL
Interface	MySQL command-line client (mysql -u root) in macOS Terminal
Database Name	travel_agency_db
Database Type	Relational Database
Main Tables	domestic_bookings, international_bookings
Advanced SQL	UNION, UNION ALL, INNER JOIN, DISTINCT, subquery, GROUP BY, COUNT(*)
Operating Environment	macOS

5. Project Workflow Diagram

The workflow below shows the order in which the project was carried out, from connecting to MySQL to verifying the final results.

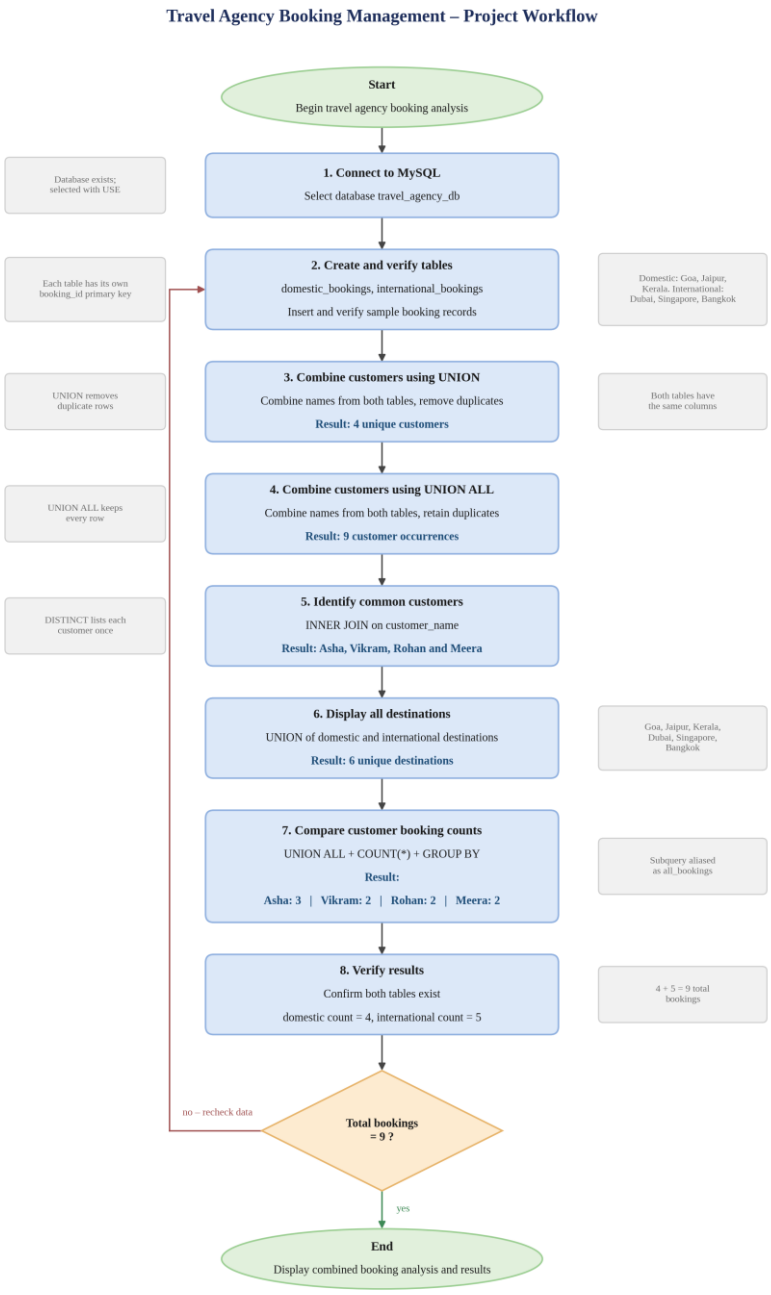


Figure 5.1: Travel Agency Booking Management – Project Workflow

6. Database Design

The database travel_agency_db contains two tables: domestic_bookings and international_bookings. Both tables have the same structure so that their query results are compatible for set operations.

6.1 domestic_bookings

Primary Key: booking_id. Attributes: booking_id, customer_name, destination, booking_date, amount. Stores the domestic bookings (Goa, Jaipur and Kerala).

Field	Type	Null	Key	Default	Extra
booking_id	INT	NO	PRI	NULL	auto_increment
customer_name	VARCHAR(100)	NO		NULL	
destination	VARCHAR(100)	NO		NULL	
booking_date	DATE	NO		NULL	
amount	DECIMAL(10,2)	NO		NULL	

6.2 international_bookings

Primary Key: booking_id. Attributes: booking_id, customer_name, destination, booking_date, amount. Stores the international bookings (Dubai, Singapore and Bangkok).

Field	Type	Null	Key	Default	Extra
booking_id	INT	NO	PRI	NULL	auto_increment
customer_name	VARCHAR(100)	NO		NULL	
destination	VARCHAR(100)	NO		NULL	
booking_date	DATE	NO		NULL	
amount	DECIMAL(10,2)	NO		NULL	

6.3 Relationship between the tables

The two tables are independent. Each table has its own booking_id primary key and there is no foreign key between them. The same customer names appear in both tables, and they are matched only inside queries (UNION, UNION ALL and INNER JOIN on customer_name).

7. ER Diagram

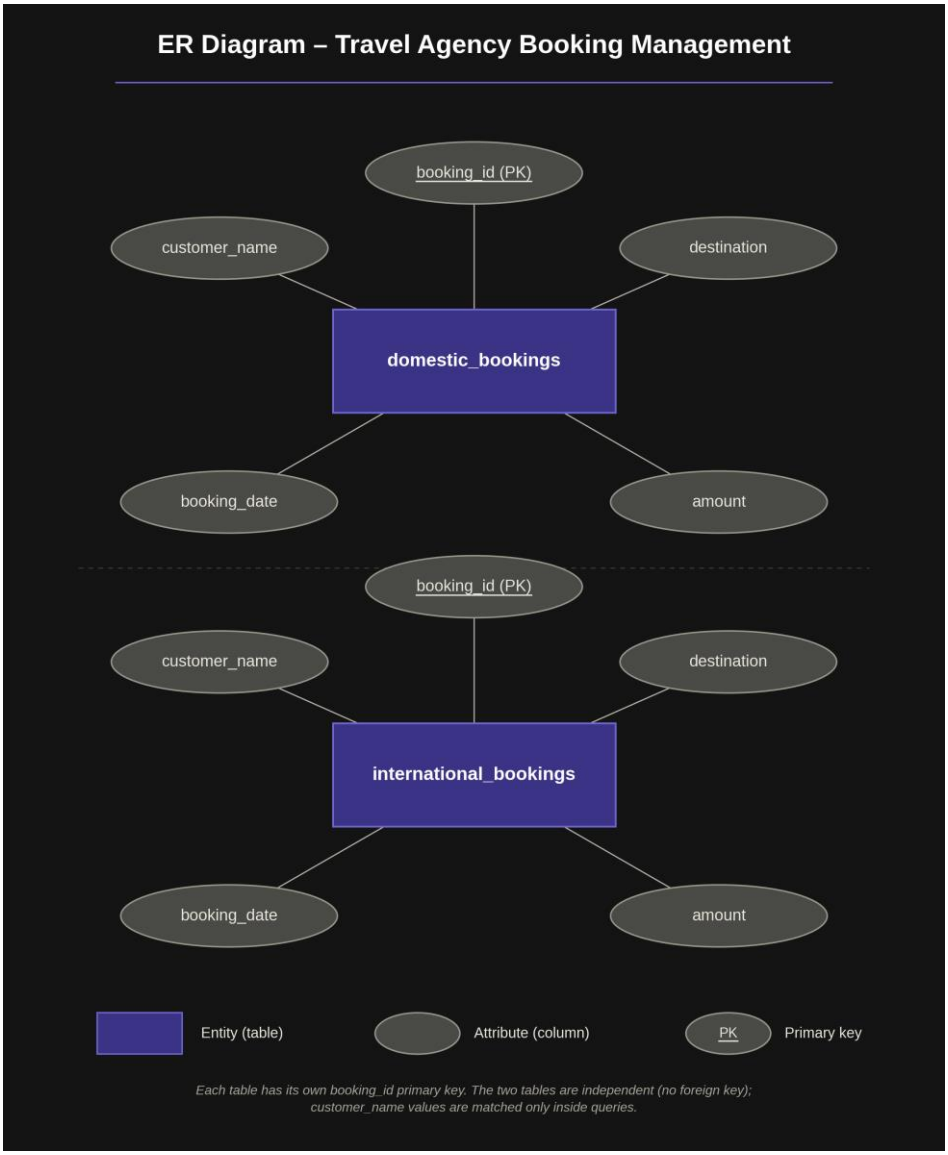


Figure 7.1: ER Diagram – Travel Agency Booking Management

8. Relational Schema

domestic_bookings (booking_id PK, customer_name, destination, booking_date, amount)

international_bookings (booking_id PK, customer_name, destination, booking_date, amount)

There are no foreign keys in this schema.

9. DDL – Table Creation

The database and tables were created using SQL Data Definition Language (DDL). Each table uses a PRIMARY KEY with AUTO_INCREMENT on booking_id, and all other columns are defined as NOT NULL. The CREATE DATABASE statement reported that travel_agency_db already existed, so the database was selected with USE.

9.1 Database Selection and Table Verification

```
mysql> USE travel_agency_db;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> SHOW TABLES;
+----------------------------+
| Tables_in_travel_agency_db |
+----------------------------+
| domestic_bookings          |
| international_bookings     |
+----------------------------+
2 rows in set (0.001 sec)
```

Figure 9.1: USE travel_agency_db and SHOW TABLES

9.2 domestic_bookings Table Structure

```
mysql> DESCRIBE domestic_bookings;
+---------------+---------------+------+-----+---------+----------------+
| Field         | Type          | Null | Key | Default | Extra          |
+---------------+---------------+------+-----+---------+----------------+
| booking_id    | int           | NO   | PRI | NULL    | auto_increment |
| customer_name | varchar(100)  | NO   |     | NULL    |                |
| destination   | varchar(100)  | NO   |     | NULL    |                |
| booking_date  | date          | NO   |     | NULL    |                |
| amount        | decimal(10,2) | NO   |     | NULL    |                |
+---------------+---------------+------+-----+---------+----------------+
5 rows in set (0.061 sec)
```

Figure 9.2: DESCRIBE domestic_bookings

The structure is equivalent to the following definition:

```
CREATE TABLE domestic_bookings (
    booking_id INT AUTO_INCREMENT PRIMARY KEY,
    customer_name VARCHAR(100) NOT NULL,
    destination VARCHAR(100) NOT NULL,
    booking_date DATE NOT NULL,
    amount DECIMAL(10,2) NOT NULL
);
```

9.3 international_bookings Table Structure

```
mysql> DESCRIBE international_bookings;
+--------------+--------------+------+-----+---------+----------------+
| Field        | Type         | Null | Key | Default | Extra          |
+--------------+--------------+------+-----+---------+----------------+
| booking_id   | int          | NO   | PRI | NULL    | auto_increment |
| customer_name| varchar(100) | NO   |     | NULL    |                |
| destination  | varchar(100) | NO   |     | NULL    |                |
| booking_date | date         | NO   |     | NULL    |                |
| amount       | decimal(10,2)| NO   |     | NULL    |                |
+--------------+--------------+------+-----+---------+----------------+
5 rows in set (0.002 sec)
```

Figure 9.3: DESCRIBE international_bookings

The structure is equivalent to the following definition:

```
CREATE TABLE international_bookings (
    booking_id INT AUTO_INCREMENT PRIMARY KEY,
    customer_name VARCHAR(100) NOT NULL,
    destination VARCHAR(100) NOT NULL,
    booking_date DATE NOT NULL,
    amount DECIMAL(10,2) NOT NULL
);
```

10. DML – Sample Data

Sample records were inserted into both tables. The domestic table contains four bookings and the international table contains five bookings.

10.1 domestic_bookings Table Output

```
mysql> SELECT * FROM domestic_bookings;
+------------+---------------+-------------+--------------+----------+
| booking_id | customer_name | destination | booking_date | amount   |
+------------+---------------+-------------+--------------+----------+
|          1 | Asha          | Goa         | 2026-01-10   | 15000.00 |
|          2 | Vikram        | Jaipur      | 2026-01-15   | 12000.00 |
|          3 | Rohan         | Kerala      | 2026-02-05   | 18000.00 |
|          4 | Meera         | Goa         | 2026-02-20   | 14000.00 |
+------------+---------------+-------------+--------------+----------+
4 rows in set (0.002 sec)
```

*Figure 10.1: SELECT * FROM domestic_bookings*

10.2 international_bookings Table Output

```
mysql> SELECT * FROM international_bookings;
+------------+---------------+-------------+--------------+----------+
| booking_id | customer_name | destination | booking_date | amount   |
+------------+---------------+-------------+--------------+----------+
|          1 | Asha          | Dubai       | 2026-03-10   | 45000.00 |
|          2 | Vikram        | Singapore   | 2026-03-15   | 55000.00 |
|          3 | Rohan         | Bangkok     | 2026-04-05   | 40000.00 |
|          4 | Meera         | Dubai       | 2026-04-20   | 48000.00 |
|          5 | Asha          | Singapore   | 2026-05-01   | 52000.00 |
+------------+---------------+-------------+--------------+----------+
5 rows in set (0.000 sec)
```

*Figure 10.2: SELECT * FROM international_bookings*

Asha appears once in the domestic table and twice in the international table (Dubai and Singapore), which creates the duplicate customer entries used in the UNION and UNION ALL comparison.

11. Normalization

Normalization is used to reduce data redundancy and prevent insertion, update and deletion anomalies.

First Normal Form – 1NF

- Each column holds a single atomic value and there are no repeating groups.
- Every row is uniquely identified by its booking_id primary key.

Second Normal Form – 2NF Each table has a single-column primary key, so no non-key attribute can depend on only part of the key. Both tables are in 2NF.

Third Normal Form – 3NF The non-key attributes (customer_name, destination, booking_date and amount) describe the booking itself and do not determine one another, so no transitive dependency is present within a table.

Note: Customer names are stored as text in each booking row because the case study keeps the two booking datasets separate and does not define a customer table. A separate customer table could be added in future to remove this repetition.

12. SQL Queries and Outputs

12.1 Combine Customers using UNION

UNION combines the customer names of both tables and removes duplicate rows.

```
mysql> SELECT customer_name
    -> FROM domestic_bookings
    -> UNION
    -> SELECT customer_name
    -> FROM international_bookings;
+---------------+
| customer_name |
+---------------+
| Asha          |
| Vikram        |
| Rohan         |
| Meera         |
+---------------+
4 rows in set (0.004 sec)
```

Figure 12.1: UNION – 4 unique customers

Result: 4 unique customers – Asha, Vikram, Rohan and Meera.

12.2 Preserve Duplicates using UNION ALL

UNION ALL combines the customer names of both tables and keeps every row, including duplicates.

```
mysql> SELECT customer_name
    -> FROM domestic_bookings
    -> UNION ALL
    -> SELECT customer_name
    -> FROM international_bookings;
+---------------+
| customer_name |
+---------------+
| Asha          |
| Vikram        |
| Rohan         |
| Meera         |
| Asha          |
| Vikram        |
| Rohan         |
| Meera         |
| Asha          |
+---------------+
9 rows in set (0.001 sec)
```

Figure 12.2: UNION ALL – 9 customer occurrences

Result: 9 customer occurrences (4 domestic + 5 international). Asha appears three times.

12.3 Customers Appearing in Both Datasets (INNER JOIN)

An INNER JOIN on customer_name finds customers present in both tables. DISTINCT is used because Asha has two international bookings and would otherwise be repeated.

```
mysql> SELECT DISTINCT d.customer_name
    -> FROM domestic_bookings d
    -> INNER JOIN international_bookings i
    -> ON d.customer_name = i.customer_name;
+---------------+
| customer_name |
+---------------+
| Asha          |
| Vikram        |
| Rohan         |
| Meera         |
+---------------+
4 rows in set (0.002 sec)
```

Figure 12.3: INNER JOIN – customers in both tables

Result: Asha, Vikram, Rohan and Meera appear in both the domestic and international datasets.

12.4 Display All Destinations in a Single Report

UNION combines the destinations of both tables into one list without duplicates (Goa appears twice in the domestic table and Dubai and Singapore appear twice in the international table).

```
mysql> SELECT destination
    -> FROM domestic_bookings
    -> UNION
    -> SELECT destination
    -> FROM international_bookings;
+-------------+
| destination |
+-------------+
| Goa         |
| Jaipur      |
| Kerala      |
| Dubai       |
| Singapore   |
| Bangkok     |
+-------------+
6 rows in set (0.001 sec)
```

Figure 12.4: UNION – 6 unique destinations

Result: 6 unique destinations – Goa, Jaipur, Kerala, Dubai, Singapore and Bangkok.

12.5 Compare Customer Booking Counts

UNION ALL is placed inside a derived table (all_bookings) so that every booking is kept. COUNT(*) and GROUP BY then count the bookings of each customer.

```
mysql> SELECT
    ->     customer_name,
    ->     COUNT(*) AS booking_count
    -> FROM (
    ->     SELECT customer_name
    ->     FROM domestic_bookings
    ->     UNION ALL
    ->     SELECT customer_name
    ->     FROM international_bookings
    -> ) AS all_bookings
    -> GROUP BY customer_name;
+---------------+---------------+
| customer_name | booking_count |
+---------------+---------------+
| Asha          |             3 |
| Vikram        |             2 |
| Rohan         |             2 |
| Meera         |             2 |
+---------------+---------------+
4 rows in set (0.003 sec)
```

Figure 12.5: Booking count per customer

Result: Asha = 3, Vikram = 2, Rohan = 2 and Meera = 2.

13. UNION vs UNION ALL

Feature	UNION	UNION ALL
Duplicate rows	Removed	Preserved
Customer query result	4 rows	9 rows
Typical use in this project	Unique customers and destinations	Counting every booking per customer
Requirement	Same number of columns and compatible data types	Same number of columns and compatible data types

The difference between the two customer results (9 rows with UNION ALL and 4 rows with UNION) shows that UNION removed 5 duplicate customer entries.

14. Final Database Verification

14.1 Current Database

```
mysql> SELECT DATABASE();
+------------------+
| DATABASE()       |
+------------------+
| travel_agency_db |
+------------------+
1 row in set (0.000 sec)
```

Figure 14.1: SELECT DATABASE()

14.2 domestic_bookings Record Count

```
mysql> SELECT COUNT(*) AS domestic_count
    -> FROM domestic_bookings;
+----------------+
| domestic_count |
+----------------+
|              4 |
+----------------+
1 row in set (0.003 sec)
```

Figure 14.2: Domestic booking count

14.3 international_bookings Record Count

```
mysql> SELECT COUNT(*) AS international_count
    -> FROM international_bookings;
+---------------------+
| international_count |
+---------------------+
|                   5 |
+---------------------+
1 row in set (0.001 sec)

mysql>
```

Figure 14.3: International booking count

14.4 Verified Results

Check	Verified result
Database	travel_agency_db
Tables	domestic_bookings, international_bookings
Domestic bookings	4
International bookings	5

Check	Verified result
Total bookings	9
Unique customers (UNION)	4
Customer occurrences (UNION ALL)	9
Customers in both tables	Asha, Vikram, Rohan and Meera
Unique destinations	6
Booking counts	Asha = 3, Vikram = 2, Rohan = 2, Meera = 2

15. Objectives and Results

Objective	Query used	Result
Combine customers	UNION	4 unique customers
Preserve duplicates	UNION ALL	9 occurrences
Customers in both datasets	INNER JOIN with DISTINCT	4 customers
All destinations in one report	UNION	6 destinations
Compare booking counts	UNION ALL + COUNT(*) + GROUP BY	Asha 3; others 2

16. Data Integrity

Data integrity means maintaining accurate and consistent information in the database. This project maintains it through:

- PRIMARY KEY on booking_id, which uniquely identifies every booking in each table
- AUTO_INCREMENT, which generates unique booking_id values automatically
- NOT NULL constraints on every column, so no booking is stored with missing information
- Fixed data types (INT, VARCHAR(100), DATE and DECIMAL(10,2)) that keep the data valid

The two tables are independent, so no foreign key constraints are defined between them.

17. Advantages of the System

- Keeps domestic and international bookings separate while still allowing combined reports.
- Set operators produce combined customer and destination reports without merging the tables.
- UNION and UNION ALL give both unique and complete views of the same data.
- Booking counts per customer help management compare customer activity.
- Constraints and verification queries keep the data accurate.
- The queries are short, readable and easy to reuse.

18. Limitations

The current academic implementation focuses on the SQL set operations required by the case study. It has the following limitations:

- There is no separate customer table, so customers are identified only by name; two different customers with the same name would be treated as one customer.
- The two tables have no foreign key relationship.
- Only a small sample data set is used.
- The project does not include payment processing, user authentication or a user interface.
- The common-customer query uses INNER JOIN. Newer MySQL versions also support INTERSECT, but the join works in every version.

19. Future Scope

- Add a customer table with a customer_id and link bookings to it using foreign keys.
- Create a combined view of domestic and international bookings.
- Add revenue reports using SUM(amount) and date-wise reports using booking_date.
- Add more destinations, customers and booking types.
- Build a web-based or mobile booking interface.
- Add indexes for faster queries on larger data sets.
- Use NoSQL storage for selected high-volume application data.

20. Conclusion

The Travel Agency Customer and Booking Database demonstrates how a relational database can keep domestic and international bookings in separate tables and still produce combined reports. The project uses MySQL and SQL to create the domestic_bookings and international_bookings tables with primary keys and NOT NULL constraints.

UNION combined the customers into 4 unique names, while UNION ALL preserved all 9 customer occurrences. An INNER JOIN showed that Asha, Vikram, Rohan and Meera appear in both datasets, a UNION query listed the 6 unique destinations, and UNION ALL with COUNT(*) and GROUP BY showed booking counts of 3 for Asha and 2 each for Vikram, Rohan and Meera. The verification queries confirmed 4 domestic bookings, 5 international bookings and 9 bookings in total.